

Unlocking a New Rust Programming Experience: Fast and Slow Thinking with LLMs to Conquer Undefined Behaviors

Renshuang Jiang^{†¶}, Pan Dong^{†¶}, Zhenling Duan[†], Yu Shi[†], Xiaoxiang Fang[†], Yan Ding[†],
Jun Ma[†], Shuai Zhao[‡], Zhe Jiang^{§*}

[†]National University of Defense Technology, China. [‡]Sun Yat-sen University, China. [§]Southeast University, China.

Abstract—To provide flexibility and low-level interaction capabilities, the “unsafe” tag in Rust is essential, but undermines memory safety and introduces Undefined Behaviors (UBs) that reduce safety. Eliminating UBs requires a deep understanding of Rust’s safety rules and strong typing. Traditional methods require depth analysis of code, which is laborious and depends on knowledge design. The powerful semantic understanding capabilities of LLM offer new opportunities to solve this problem. Although existing large model debugging frameworks excel in semantic tasks, limited by fixed processes and lack adaptive and dynamic adjustment capabilities. Inspired by the dual process theory of decision-making (“Fast and Slow Thinking”), we present a LLM-based framework called RustBrain that automatically and flexibly minimizes UBs in Rust projects. Fast thinking extracts features to generate solutions, while slow thinking decomposes, verifies, and generalizes them abstractly. To apply verification and generalization results to solution generation, enabling dynamic adjustments and precise outputs, RustBrain integrates two thinking through a feedback mechanism. Experimental results on Miri dataset show a 94.3% *pass* rate and 80.4% *execution* rate, improving flexibility and Rust projects safety.

I. INTRODUCTION

Rust, a systems programming language, is popular for its guarantees of memory safety and concurrency [1], [2]. For flexibility, Rust has a secondary language hidden inside it which does not enforce memory safety guarantees: it’s called Unsafe Rust and works just like regular Rust (it’s called Safe Rust). Safe Rust ensures safety through strict compiler checks. Unsafe Rust uses *unsafe* keyword to grant direct control over memory and system resources, bypassing certain compiler checks, with safety guarantees that depend on the developer. Therefore, this capability introduces the potential for Undefined Behaviors (UBs), a class of run-time bugs that are difficult to detect and can lead to severe security vulnerabilities and instability [3], [4]. Since Unsafe Rust cannot be avoided [5], especially in system software, how to perform fixes is a key in ensuring Rust’s safety.

The main methods for fixing UBs in Unsafe Rust include safe substitution, inserting assertions, and semantic modification [6]–[9]. Rust’s strongly typed system, along with its strict ownership, lifetime, and borrowing rules, makes this process challenging, requiring an in-depth understanding of complex code semantics [40]. Although the types of Unsafe Rust operations are finite (five types of unsafe operation [1], [5]), minor semantic variation significantly affect how code modification is approached [3], [9], leading to situations where similar UB may not yield comparable solutions. Therefore, understanding semantics and possessing specialized knowledge are essential to addressing Rust safety issues. Traditional methods [33]–[37] require engineers to apply specialized knowledge along with strict logical reasoning, judgment, and verification processes, where the efficiency of resolution is closely tied to experience and expertise. The powerful self-reasoning and semantic understanding capabilities of large language models (LLMs) offer new opportunities to enhance the safety of Rust programs [10]–[13]. Due to high training costs and a lack of high-quality data in specific domains, conventional LLMs struggle with deep understanding and complex relational reasoning when handling specialized languages like Rust. In contrast, existing

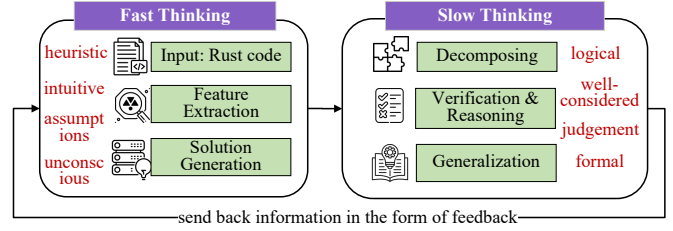


Fig. 1: RustBrain: Fast and Slow thinking LLM processes.

large model repair frameworks [12], [13], while providing assistance for such problems, typically follow pre-designed and fixed repair processes, lacking the professional capabilities to tailor solutions based on the specific characteristics of the code.

Inspired by human cognitive psychology, “*Thinking Fast and Slow*” [14], we observe that LLMs provide a broad, generalized understanding that resembles the “fast thinking” of humans. This thinking is rapid and intuitive, but it is also error-prone reasoning. Yet, unlike human cognition, LLMs lack a “slow thinking” mechanism, employing specialized knowledge and logical reasoning to enhance accuracy, identify discrepancies, and mitigate errors. This is crucial when addressing complex problems that demand deliberate and systematic approaches to detect inaccuracies and ensure correctness. These two complementary processes – fast, intuitive reasoning and slow, analytical reasoning – form a holistic framework for understanding and solving complex problems, e.g., solving UBs in Rust.

In light of this, we introduce RustBrain, a framework that integrates these cognitive processes, leveraging their respective strengths to autonomously and flexibly minimize UBs. As depicted in Fig. 1, in the fast-thinking process, it extracts code information, identifies error types, and analyzes them using a broad knowledge base to generate multiple repair solutions. In the slow-thinking process, the framework in-depth analyzes and validates the repair solutions based on the four design principles: decomposition, verification, reasoning and generalization. First, the solutions are decomposed and executed using multiple agents. In this stage, based on observations of Rust, we designed several agents, including three types of error-fixing agents, adaptive rollback agent, and abstract reasoning agent, to support better decomposition and verification. Then, the solution is fine-tuned based on verification results in slow-thinking, further optimizing the solution. Finally, to enhance the framework’s generalization capability and better support precise solutions for similar errors, RustBrain establishes an evaluation and self-learning mechanism. This feeds the execution and evaluation results from the slow-thinking stage back into the fast-thinking, continuously optimizing solution generation and refining decision-making.

To evaluate RustBrain, we utilize a dataset containing UBs provided by Miri repository [17]. The results demonstrate that it achieves 94% *pass* rate and 80% execution semantic acceptability rate. Compared to the state-of-the-art Rust fix tool (RustAssistant [13]), it *pass* rate has increased by 30%. Additionally, compared to engineer experts, the speed evaluation shows an improvement of up to 18x.

[¶] Both authors contributed equally to this research.

^{*} represents corresponding authors. Emails: zhejiang.uk@gmail.com.

II. RUSTBRAIN: AN OVERVIEW

A. Scope

Our goal is to build a toolchain that flexibly and systematically evaluates the capabilities of RustBrain for eliminating UBs in Rust at compile time. Repaired code must pass the Miri compiler while preserving its expected semantics. Miri [17] is a UB detection tool for Rust. Since many Rust code repairs are closely tied to context, the acceptability of semantics before and after the repair may vary significantly. Therefore, this paper validates semantics using test benchmarks composed of developer-repaired code. Compared to direct testing through Miri, this semantic judgment approach may introduce a degree of subjectivity. This is due to factors such as the size of the benchmark (i.e., the probability of covering all possible semantics) and other variables. Besides, we use Miri to detect Rust UBs and only considers Rust source files (i.e., files with .rs extension). Errors not detected by Miri are beyond the scope of this paper. In addition, RustBrain uses LLMs to guide the reduction of UBs, and we temporarily disregard Rust code that exceeds LLM tokens limits.

B. RustBrain Overview

According to Kahneman’s theory [14], human decision-making relies on two main processes: fast thinking, which handles intuitive, quick, often unconscious decisions, and slow thinking, which tackles complex situations through logical reasoning and careful analysis for well-considered choices. These are suitable for solving highly specialized problems that require comprehensive analysis and precise judgment. We observe that unsafe operations in Rust are limited and rely on intuitive, pattern-driven processes, similar to fast thinking. However, Rust’s strong type system requires specialized verification and improvements, similar to slow thinking. Therefore, RustBrain is designed as a dual-phase thinking framework incorporating fast and slow thinking (see Fig. 2). Fast thinking is intuitive and rapidly processes tasks, generating potential repair solutions based on code features which include error identification, code feature extraction. Slow thinking provides deliberate and in-depth processing. It combines decomposition, validation, reasoning and generalization to better understand the essence of the problem, delivering more targeted and comprehensive solutions for these types of UBs. By leveraging these two types of thinking, the framework achieves adaptive and high-precision repairs, improving both efficiency and reliability in eliminating UBs. RustBrain framework involves the following stages:

- **Stage F1 and F2:** During fast thinking process, Rust code is used as input, and Miri is utilized to detect UBs. If no UBs are found, the process terminates, outputting “pass”. Conversely, relevant errors, code, and design hints are sent to a LLM agent for feature extraction. Subsequently, these features are integrated with another LLM to rapidly generate multiple repair solutions.
- **Stage S1:** These solutions are decomposed, validated, reasoned and generalized during the slow thinking process. Each solution is broken down into refined steps to enable distributed processing and resolution by different agents. The order of these steps reflects diverse repair strategies, enabling flexible process combinations to optimize repair paths and outcomes.
- **Stage S2:** This stage integrates multiple agents to verify solutions, including agents designed in previous work [27], [41], and new agents specifically developed based on Rust’s unique characteristics. These agents include: (i) safe-replacement, assertion-adding, and code-modification agents fix UBs to improve code safety. (ii) adaptive rollback agent to reduce hallucinations and propagation in LLMs by reverting to the optimal code state; (iii) abstract reasoning agent supports deep semantic understanding

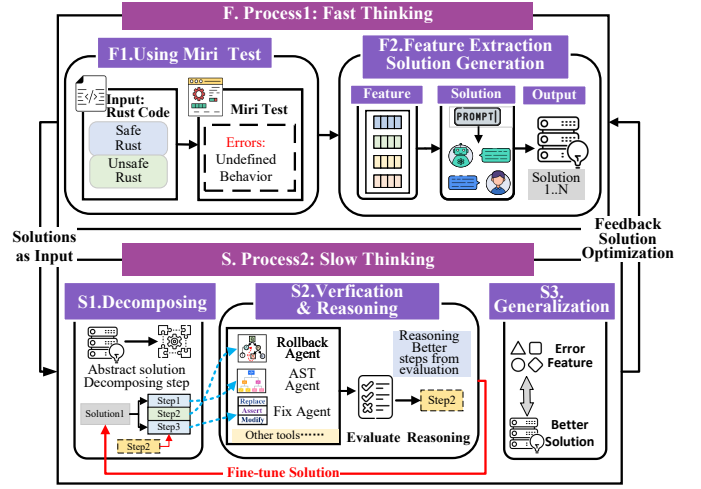


Fig. 2: RustBrain Overview: it leverages Miri for static analysis (F1). Extracts and combines features for solution generation (F2). Abstraction and decomposition into steps (S1), integrates multi-agent verification to reduce UBs. Evaluate and reason to fine-tune solution (S2). Generalization provides a solution for such UBs(S3).

and provides more professional fixes. Additionally, the evaluation mechanism is designed to conduct multidimensional assessments of solutions and further refine them through reasoning. For instance, increasing the iteration count, adjusting parameters in the repair steps, optimizing the execution path, etc.

- **Stage S3:** To provide precise solutions for similar errors, we design a feedback mechanism to match error features with optimized solutions, combining in-depth evaluation of slow thinking with generation of fast thinking. This self-learning method integrates both stages’ strengths, reduces reliance on knowledge, and enables dynamic solution optimization.

Overall, RustBrain integrates fast and slow thinking to eliminate Rust-UBs, overcoming the limitations of traditional repair frameworks. It also combines decomposition, verification, reasoning, and generalization to achieve deep semantic understanding and expert-level support, providing more precise solutions for similar errors while reducing reliance on knowledge bases.

III. RUSTBRAIN IMPLEMENTATION

This section explores the technical implementation of the “Fast and Slow Thinking” framework. First, we classify five types of unsafe Rust operations and design the fast thinking process for generating solutions (Section III-A). Then, we present the slow thinking process to decompose solutions and execute them with multi-agents: agents for errors fixing with LLMs (Section III-B1); agents of rollback mechanisms and optimal code selection (Section III-B2) to mitigate hallucinations; and abstract reasoning agents leveraging Rust semantics for expert guidance (Section III-B3). Finally, the results from the slow-thinking are feedback into the fast-thinking to refine evaluations and generate more specialized solutions (Section III-C).

A. Classification of Unsafe Rust and Fast Thinking Process

To match the error types as closely as possible with the solutions generated by the fast thinking process, we further classify Unsafe Rust from the perspective of error repair. This classification allows for targeted problem-solving by identifying the unique characteristics and challenges of each type of unsafe operation.

- 1) **Classification of Unsafe Rust:** Rust’s unique design ensures that programs written in Safe Rust have no memory safety issues [5], [22]. However, Unsafe Rust allows programmers to bypass these

safety checks and perform certain operations. Unlike C, where unsafe operations are everywhere and are barely checked by the compiler [23], we observe that Rust explicitly classifies unsafe operations and limits them to “unsafe” code blocks. Even errors arising from interactions between unsafe and safe code occur within the “unsafe” block. This design significantly reduces the surface area for potential memory safety bugs, thereby facilitating error localization.

Unsafe Rust can be divided into five categories [1]: dereference raw pointers, call unsafe functions, implement unsafe traits, access/modify mutable static variables, and access union fields. Based on experience and research [4], [9], we further categorize Unsafe Rust from the perspective of error fixing as follows: (i) **Operations with Safe Alternatives**: Certain unsafe operations can be replaced with safer APIs from Rust’s standard library. (ii) **Operations Checking via Assertion**: Adding assertions helps detect boundary and type violations, enhancing safety without fundamentally altering the program’s structure. (iii) **Operations Semantic Modification**: Understanding the underlying intentions of the original code, modifying erroneous semantics, and ensuring that these intentions are preserved in the modifications. Given Rust’s strong type system, we observe that similar unsafe operations often require distinct handling methods. For instance, in case (i), as shown in Fig 3, the same unsafe API may require different replacement APIs depending on the context.

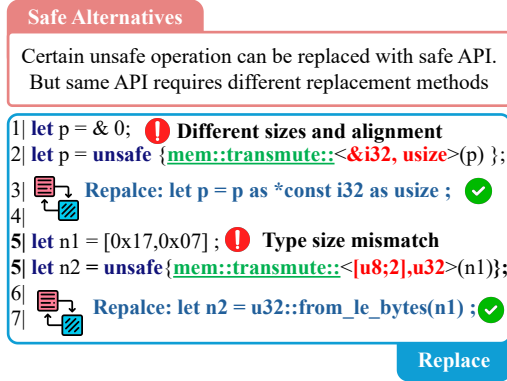


Fig. 3: The same API requires different substitution methods.

Therefore, based on this classification, the following principles are proposed in this paper to guide the framework design:

- **Principle 1**: All Unsafe Rust operations are explicitly marked with the “unsafe” keyword, reducing the surface area for potential memory safety vulnerabilities facilitates error localization.
- **Principle 2**: The limited types of UBs caused by Unsafe Rust can be further classified into three categories: replaced by Safe Rust, prevented by assertions and require semantic modification.
- **Principle 3**: Under Rust’s strong typing characteristics, a single piece of code may exhibit multiple patterns, necessitating that changes be combined with semantic analysis.

2) **Fast Thinking Process**: Undefined behaviors usually occur at runtime. We believe that *prevention is better than cure*, therefore RustBrain integrates tools such as Miri compiler to analyze the Rust code. This static analysis occurs at compile time, where the compiler identifies potential UBs within the code, and records them for further examination. Once the detection phase is complete, the framework enters the fast thinking process.

In this process, RustBrain quickly identifies and generates fix solutions based on intuitive Unsafe Rust error pattern recognition. Firstly, RustBrain, based on five types of unsafe operations, employs a general-knowledge LLM to rapidly analyze code features, leveraging its generalization capability to identify typical error patterns and guide solution generation. According to **Principle 1**, UBs are

generally confined to unsafe regions, which narrows the scope of code analysis and reduces the noise impact of irrelevant code on feature extraction. Then, since Unsafe Rust operations are limited in types, even with strong typing considerations, similar repair solutions can typically be found for most issues. Based on this, RustBrain generates multiple repair solutions tailored to different code characteristics by integrating **Principle 2**, ensuring that the repair methods are both adaptable and diverse in addressing the unique traits of the code.

B. Slow Thinking Process

After fast thinking generates preliminary solutions, RustBrain transitions into slow thinking for deeper thinking. Due to **Principle 3**, we need to further decompose, validate, infer, and generalize solutions. First, RustBrain splits solutions into multiple steps, analyzing and addressing UBs layer by layer. Then, it integrates agents design to validate and evaluate each generated solution’s compatibility with type and safety requirements. We introduce new agents based on Rust features to support deep and professional fixes.

1) **Three Types of Error Fixing Agents**: To better guide LLM in fixing UBs errors, inspired by **Principle 2**, RustBrain constructs three types of special agents: equivalent replacement, assertion and modification. We observe that unsafe Rust is often misused, with some cases where safe code could achieve the same functionality [3], [4], [9]. Therefore, RustBrain designs an agent to identify situations where safe alternatives can replace unsafe operations. This minimizes unsafe code usage, enhancing the overall safety and reliability of Rust. Besides, we design an assertion agent that adds assertions in appropriate places to capture potential UBs at runtime before they can propagate. This avoids altering the code’s semantics while detecting and stopping potential UBs during execution in a timely manner, preventing it from leading to more severe security vulnerabilities. However, in certain complex scenarios, UBs involve deeper logical errors or dependencies on external states, and assertions alone cannot directly resolve these issues. To address this, RustBrain introduces a modification agent, where adjust the erroneous semantics in the Rust code, ensuring that the code’s functionality remains unchanged while effectively reducing the risk of UBs. Prompts are shown in the Fig. 4.

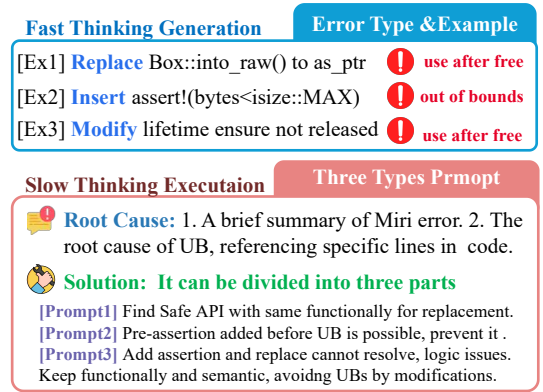


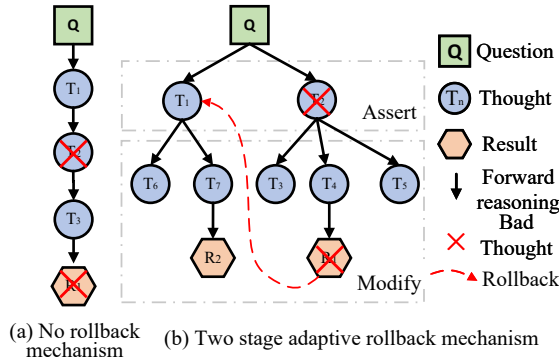
Fig. 4: Three types of prompt strategies.

2) **Adaptive Rollback and Optimal Code Selection Agent**: Using multiple fix agents can significantly reduce UBs in Rust code (Section IV). However, we observe that in some cases, the number of errors increases after repair, and analyzed and discussed this.

We define two sequences: $T = \{T_0, T_1, \dots, T_p\}$ for thoughts generated by slow thinking, and $N = \{n_0, n_1, \dots, n_p\}$ for errors detected by Miri, where T_i represents thoughts, n_i the errors count, and p the iteration count. During the slow thinking, the error sequence N may continue to grow, as in the example sequence $N_1 = \{1, 3, 4, 6, 9\}$. This phenomenon is known as “large model

hallucination" [24], [25]. As shown in Fig. 5 (a), without rollback mechanism, the process would proceed from the current error stage to the next stage, exacerbating the impact of hallucinations. Existing debugging frameworks [26], [27] typically rollback directly to the initial state T_0 to eliminate accumulated errors and mitigate the propagation of incorrect fixes. However, this discards valuable partial corrections made during the iteration process and creates a large overhead ($c * T_n$, where c is the number of rollbacks) [26].

We observe another type of error sequence, such as $N_2 = \{3, 1, 5, 2, 0\}$. This indicates that while errors may increase in certain iterations, the overall trend shows a fluctuating decline. This behavior is similar to humans often providing frequent and random errors during reasoning, but eventually decrease them and converge to a solution. Therefore, we suggest that slow thinking a certain degree of self-correcting capability, allowing it to gradually converge towards the correct through multiple iterations.



supports similar errors repairs, reduces dependency on the knowledge base, and avoids the limitations posed by highly specialized issues.

IV. EVALUATION

The evaluation of RustBrain designed to answer the following research questions:

- 1) **RQ1 (Flexibility):** How does RustBrain flexibility affect repair *pass* rate and performance?
- 2) **RQ2 (Accuracy):** What extent is RustBrain successful in fixing Rust UBs errors as measured by metrics of *pass* and *exec*?
- 3) **RQ3 (Sensitivity):** How does LLM configurations in GPT-4 impact the performance of RustBrain?
- 4) **RQ4 (Advancement):** How does the effectiveness and performance of RustBrain in eliminating UBs compared to the state-of-the-art method and human expert?

Firstly, we give a brief explanation of the datasets, large language models, baselines and metrics used in this paper.

Datasets. We use a dataset collected from the official Miri [17], containing code with UBs. The dataset is categorized into different types, including alloc, dangling pointers, and concurrency issues.

LLMs. We use four LLMs: GPT-3.5 [18], GPT-4 [19], GPT-O1 [20], and Claude 3.5 [21], each with a default temperature of 0.5.

Baselines. Apart from the Miri test, the comparison includes state-of-the-art approaches such as human expert repair (Thetis-Lathe, 2023) [3], and a LLM-based fix method (RustAssistant, 2024) [13].

Metrics. All experiments report *pass* rate(%) and *execution* rate(%), among which *pass* rate refers to the percentage of code passing Miri after repair, and *execution* rate refers to the percentage of code passing Miri with acceptable semantics.

Subsequently, we analyze the results to answer these questions.

RQ1 (Flexibility): We select a type of UB (requiring semantic modification) and input it into RustBrain. Ten solutions are generated through fast thinking, followed by reasoning and validation using slow thinking, as depicted in Fig. 7.

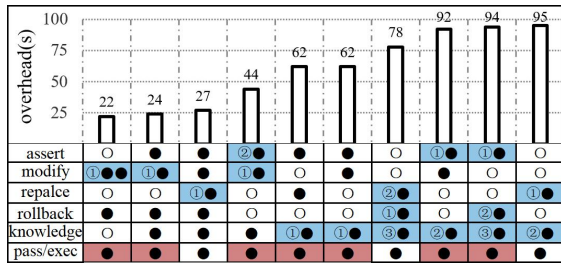


Fig. 7: RustBrain flexibly fixes UBs. Agents, shown on the left, can be enabled (●) or disabled (○), with serial numbers indicating repair order. Blue denotes agents used in execution, and red indicates semantically acceptable code, otherwise it only passes Miri.

RustBrain provides more flexible solution and improves the possible and accuracy of fixing UBs. The results indicate that: (i) For same problem, RustBrain uses fast thinking to generate diverse solutions, overcoming the limitations of fixed-process frameworks that offer only one option, enhancing flexibility and repair effectiveness. (ii) Comparing Group 1 with Groups 5, 6, 8, and 9, knowledge base can enhance problem-solving abilities, but introduce 2x-4x overhead, and depends on its size. Feedback mechanism supports more accurate solution selection, improves repair performance and reduces reliance on knowledge base. (iii) Fixed repair framework can cover a wide range of issues, it often includes numerous generic steps, which may introduce unnecessary complexity and overhead (Group 3). RustBrain dynamically designs solutions based on code features, avoiding redundant steps and improve accuracy. (iv) Specific scenarios exist for

the three types of repair solutions for UBs. Fixed repair frameworks often provide inappropriate solutions, exacerbate hallucination and reduce accuracy. In Groups 3, 7, and 10, even with knowledge base support, semantic acceptable fixes could not be achieved. RustBrain, based on code features and feedback mechanism, better guides the generation of solutions for similar error types, avoiding unsuitable fixes and improving both repair accuracy and efficiency.

RQ2 (Accuracy): We evaluated RustBrain's repair effectiveness and robustness using various configurations, including different LLMs and knowledge base usage. Experimental results, shown in Fig. 8 and Fig. 9, compare repair *pass* rates and semantic *exec* rates.

Results indicate that for GPT-4 model, integrating RustBrain achieved a *pass* rate of 90.5% and an *execution* rate of 70.2%, representing an improvement of 25-35% compared to using GPT-4 alone for repair. Furthermore, when RustBrain is integrated with the knowledge base, it achieves the highest *pass* and *execution* rates, averaging 94.3% and 80.4% , respectively. Additionally, although GPT-3.5 performed lower than GPT-4, combining it with RustBrain achieved a comparable level to GPT-4 in terms of fix *pass* and *execution* rate. This suggests that the design in RustBrain compensates for the limitations of the base model, improving the accuracy and consistency in handling UBs. For Claude-3.5, it exhibits initial semantic capabilities in reducing UBs, comparable to GPT-4. However, when combined with RustBrain, Claude-3.5 performs less effectively than GPT-4 in understanding complex dependencies and resolving issues. Compared to standalone Claude-3.5, RustBrain+Claude-3.5 improves the *pass* rate and *execution* rate by 17% and 20%, respectively.

Besides, we also compare RustBrain with the state-of-the-art GPT model, GPT O1. Due to O1's high cost, a subset of error types was analyzed, with results shown in Fig. 10. Despite GPT-O1's exceptional reasoning, its repair effectiveness remains inferior to RustBrain. For uncommon errors like *panic* errors, GPT-O1 fails to provide suitable solutions based on code features. In contrast, RustBrain, leverages the combination of fast and slow thinking to generate more targeted solutions, improving *execution* rate by 35.6%.

RQ3 (Sensitivity): Temperature is a key parameter affecting output diversity in model generation [30]. We conduct sampling tests in Miri dataset and evaluate the stability of the repair results by calculating the confidence intervals [31]. The Confidence Interval(CI) reflects the uncertainty of fixing UBs in LLMs, ensuring that the reported *pass* and *exec* rates are reliable within a certain probability range ($c=95\%$). In the experiments, we analyze the impact of different temperature settings on the performance of RustBrain in GPT-4.

As shown in Fig. 11, *pass* and *exec* rates reach highest points of 97% and 77% with a temperature of 0.5. Higher temperatures increase the flexibility of outputs. However, this flexibility comes at the cost of semantic integrity, leading to a decrease in semantic acceptability (e.g, 0.7). In contrast, lower temperatures limit flexibility and potentially missing opportunities to repair UBs.

RQ4 (Advancement): To evaluate the effectiveness and efficiency of RustBrain, compared to RustAssistant (the state-of-the-art Rust fix tool) and human experts. As shown in Fig. 12, compared to RustAssistant, the *pass* rate and *execution* rate of RustBrain increased by 33% and 41%, respectively. This indicates that RustBrain offers greater flexibility, breaking the limitations of existing repair frameworks. It is able to address issues that the current frameworks cannot solve, thereby reducing UBs and improving overall safety.

To evaluate RustBrain's effectiveness compared to human experts, we assessed their performance across various error types, as shown in Table I. While human experts excel in debugging, the framework demonstrates competitive performance in addressing UBs caused by complex semantic rules. On average, RustBrain achieves a 7x speedup across different error types, with the gap widening for more complex

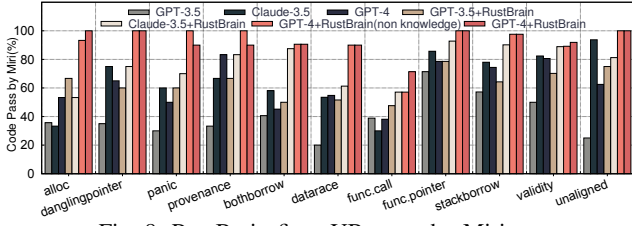


Fig. 8: RustBrain fixes UBs pass by Miri rate.

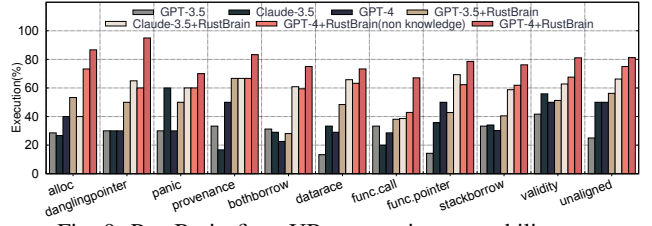


Fig. 9: RustBrain fixes UBs semantic acceptability rate.

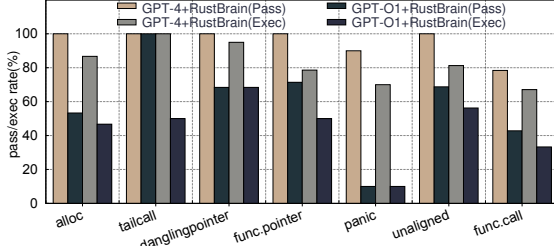


Fig. 10: Comparison of RustBrain with GPT-O1 on UBs Repair.

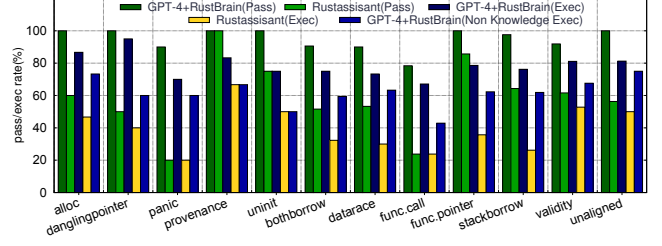


Fig. 12: Comparison of RustBrain with RustAssisat on UBs Repair.

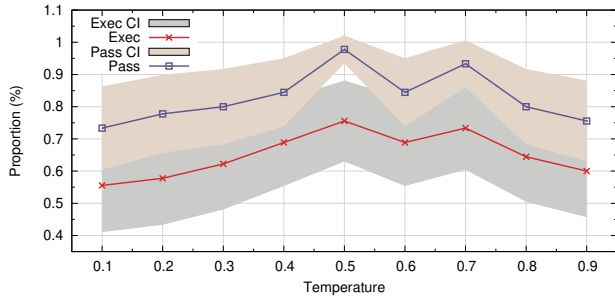


Fig. 11: Different Temperatures on RustBrain Repair Efficacy.

or rare errors. For example, in *function pointer* UBs, which mainly involve type conversion problems and require deeper Rust expertise for guidance, RustBrain shows up to 18x. Furthermore, we note that when the dataset contains similar UBs, RustBrain can reduce dependency on the knowledge base and generate precise solutions through the feedback mechanism, effectively minimizing UBs and overhead, as shown in the red sections of Table I.

Overall, for a fixed framework (such as RustAssisat), the speedup in problem-solving compared to human experts relies on the knowledge in the knowledge base, and its problem-solving ability is limited by the knowledge it contains. RustBrain leverages a feedback mechanism that feeds back evaluated solutions from the slow-thinking phase into the fast-thinking phase. Through this self-learning process, the fast-thinking can generate more precise solutions, without being fully dependent on the design of the knowledge base, reducing the overhead and time to fix UBs

V. RELATED WORK

Efforts to enhance Rust's safety focus on ensuring the safety of Unsafe Rust, categorizes as manual, semi-automated, and fully automated. Manual methods, e.g., formal methods [33], [34], static analysis [35], [36], rely on developer's expertise, enabling precise fixes but are time-consuming and error-prone. Semi-automated approaches use tools for initial error detection, followed by developers' detailed analysis and repair. Examples include symbolic execution [37], [38]. Full automated aim to minimize human involvement by leveraging machine learning [9], [39] and LLMs [12], [13]. However, its repair accuracy and coverage are limited by the knowledge base, and deep contextual understanding remains a challenge. Our approach employs a collaboration of fast and slow thinking to provide flexible and precise repair solutions while maintaining the safety of the Rust.

TABLE I: Execution Time of RustBrain against Human

Types	Average Time/s			Speedup
	GPT-4+RustBrain		Human	
	No_knowledge	knowledge		
stack borrow	86	123	366	2.81x
unaligned pointer	74	118	222	4.63x
validity	79	109	678	8.58x
alloc	40	77	450	11.25x
func. pointer	39	50	480	12.3x
provenance	45	64	240	5.33x
panic	76	107	336	4.42x
func. calls	65	74	1176	18.1x
dangling pointer	70	74	114	1.63x
both borrow	69	87	762	11x
concurrency	45	45	144	3.2x
data race	63	91	336	5.3x
Average	62.6	84.9	442	7.4x

VI. CONCLUSION

In this work, we propose a systematic, automated, and dynamically adjustable framework, called RustBrain, to eliminate UBs in Rust project. This framework combine fast and slow thinking two phases, breaking the limitations of traditional frameworks, providing more flexible fix framework, and successfully enhancing Rust code safety. The fast-thinking phase extracts code features to quickly generate multiple feasible fixes. The slow-thinking phase designs Rust feature based agents (including fixing, adaptive rollback and knowledge, etc.) to execute the solutions. The execution results feedback to the fast thinking to continuously improve and optimize the decision-making process. The experimental results show that RustBrain achieve pass rate of 94.3% and execute rate 80.4% in Miri dataset, with up to 7x speedup compared with human experts. For future work, we are interested in providing automated safety enhancements for complex Rust code involving multi-module calls.

VII. ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers for the helpful feedback. This work was supported by National Key Research and Development Program (Grant No.2024YFB4405600), the National Natural Science Foundation of China [Grant No.62472086 and No.62472432], the Pre-Research Project [Grant No.31511100101], the Science and Technology Major Special Program of Jiangsu (No. BG2024010), and the Start-up Research Fund of Southeast University (Grant No. RF1028624005).

REFERENCES

- [1] Matsakis N D, Klock F S. The rust language[J]. ACM SIGAda Ada Letters, 2014, 34(3): 103-104.
- [2] StackOverflow. <https://stackoverflow.blog/2024/07/24/developers-want-more-more-more-the-2024-results-from-stack-overflow-s-annual-developer-survey/>
- [3] Jiang R, Dong P, Ding Y, et al. Thetis: A Booster for Building Safer Systems Using the Rust Programming Language[J]. Applied Sciences, 2023, 13(23): 12738.
- [4] Qin B, Chen Y, Yu Z, et al. Understanding memory and thread safety practices and issues in real-world Rust programs[C]//Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. 2020: 763-779.
- [5] Zhu S, Zhang Z, Qin B, et al. Learning and programming challenges of rust: A mixed-methods study[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 1269-1281.
- [6] Bae Y, Kim Y, Askar A, et al. Rudra: finding memory safety bugs in rust at the ecosystem scale[C]//Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. 2021: 84-99.
- [7] Bang I, Kayondo M, Moon H, et al. TRust: A Compilation Framework for In-process Isolation to Protect Safe Rust against Untrusted Code[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 6947-6964.
- [8] Cui M, Chen C, Xu H, et al. SafeDrop: Detecting memory deallocation bugs of rust programs via static data-flow analysis[J]. ACM Transactions on Software Engineering and Methodology, 2023, 32(4): 1-21.
- [9] Dong Y, Zhang Z, Cui M, et al. SafeNet: Towards mitigating replaceable unsafe Rust code via a recommendation-based approach[J]. Software Testing, Verification and Reliability, 2024: e1875.
- [10] Zhong L, Wang Z, Shang J. Ldb: A large language model debugger via verifying runtime execution step-by-step[J]. arXiv preprint arXiv:2402.16906, 2024.
- [11] Li H, Hao Y, Zhai Y, et al. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach[J]. Proceedings of the ACM on Programming Languages, 2024, 8(OOPSLA1): 474-499.
- [12] Wu X, Cheriere N, Zhang C, et al. RustGen: An Augmentation Approach for Generating Compilable Rust Code with Large Language Models[J]. Deligiannis P, Lal A, Mehrotra N, et al. RustAssistant: Using LLMs to Fix Compilation Errors in Rust Code[C]//2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE Computer Society, 2024: 267-279.
- [14] Daniel K. Thinking, fast and slow[M]. 2017.
- [15] Booch G, Fabiano F, Horesh L, et al. Thinking fast and slow in AI[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2021, 35(17): 15042-15046.
- [16] Ganapini M B, Campbell M, Fabiano F, et al. Thinking fast and slow in AI: The role of metacognition[C]//International Conference on Machine Learning, Optimization, and Data Science. Cham: Springer Nature Switzerland, 2022: 502-509.
- [17] Miri.<https://github.com/rust-lang/miri>.
- [18] OpenAI. GPT-3.5. <https://platform.openai.com/docs/models/gpt-3-5>.
- [19] OpenAI. GPT-4. <https://platform.openai.com/docs/models/gpt-4-turbo-and-gpt-4>.
- [20] OpenAI. GPT-O1-Preview. <https://platform.openai.com/docs/models#o1>
- [21] Claude 3.5. <https://www.anthropic.com/claude/sonnet>.
- [22] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: Securing the foundations of the Rust programming language[J]. Proceedings of the ACM on Programming Languages, 2017, 2(POPL): 1-34.
- [23] Ling M, Yu Y, Wu H, et al. In Rust we trust: a transpiler from unsafe C to safer Rust[C]//Proceedings of the ACM/IEEE 44th international conference on software engineering: companion proceedings. 2022: 354-355.
- [24] Rawte V, Sheth A, Das A. A survey of hallucination in large foundation models[J]. arXiv preprint arXiv:2309.05922, 2023.
- [25] Jiang C, Xu H, Dong M, et al. Hallucination augmented contrastive learning for multimodal large language model[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 27036-27046.
- [26] Chen S, Li B. Toward Adaptive Reasoning in Large Language Models with Thought Rollback[C]//Forty-first International Conference on Machine Learning.
- [27] Xu K, Sun J, Hu Y, et al. MEIC: Re-thinking RTL Debug Automation using LLMs[J]. arXiv preprint arXiv:2405.06840, 2024.
- [28] Feng S, Shi W, Wang Y, et al. Don't Hallucinate, Abstain: Identifying LLM Knowledge Gaps via Multi-LLM Collaboration[J]. arXiv preprint arXiv:2402.00367, 2024.
- [29] Sun K, Xu Y E, Zha H, et al. Head-to-tail: How knowledgeable are large language models (llm)? AKA will llms replace knowledge graphs?[J]. arXiv preprint arXiv:2308.10168, 2023.
- [30] Renze M, Guven E. The effect of sampling temperature on problem solving in large language models[J]. arXiv preprint arXiv:2402.05201, 2024.
- [31] Smithson M. Confidence intervals[M]. Sage, 2003.
- [32] Dewey J. Reconstruction in philosophy[M]. Beacon Press, 1971.
- [33] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: Securing the foundations of the Rust programming language[J]. Proceedings of the ACM on Programming Languages, 2017, 2(POPL): 1-34.
- [34] Astrauskas V, Müller P, Poli F, et al. Leveraging Rust types for modular specification and verification[J]. Proceedings of the ACM on Programming Languages, 2019, 3(OOPSLA): 1-30.
- [35] Li Z, Wang J, Sun M, et al. MirChecker: detecting bugs in Rust programs via static analysis[C]//Proceedings of the 2021 ACM SIGSAC conference on computer and communications security. 2021: 2183-2196.
- [36] Cui M, Chen C, Xu H, et al. SafeDrop: Detecting memory deallocation bugs of rust programs via static data-flow analysis[J]. ACM Transactions on Software Engineering and Methodology, 2023, 32(4): 1-21.
- [37] Lindner M, Fittinghoff N, Eriksson J, et al. Verification of safety functions implemented in Rust-a symbolic execution based approach[C]//2019 IEEE 17th International Conference on Industrial Informatics (INDIN). IEEE, 2019, 1: 432-439.
- [38] Lindner M, Aparicius J, Lindgren P. No panic! Verification of Rust programs by symbolic execution[C]//2018 IEEE 16th International Conference on Industrial Informatics (INDIN). IEEE, 2018: 108-114.
- [39] Lu X, Zhang Z, Xu H. Safe Rust Code Recommendation Based on Siamese Graph Neural Network[C]//2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC). IEEE, 2022: 1-11.
- [40] Jung R. Understanding and evolving the Rust programming language[J]. 2020.
- [41] Kang S, Chen B, Yoo S, et al. Explainable automated debugging via large language model-driven scientific debugging[J]. arXiv preprint arXiv:2304.02195, 2023.